Journal notes on:
Miscellaneous topics from my PhD

by Francisco Lobo

I. MISCELLANEOUS NUMERICAL METHODS

A. Numerical tools for many-body electron interactions

We now make a short overview of some of the commonly used approaches to theoretically study excitonic physics and the main struggles of trying to account for the many-body electron interactions.

We start by mentioning the workhorse of ground-state mean-field electronic properties, known as *density-functional theory* (DFT), belonging to the family of first principles (ab initio) methods. DFT provides a way to map the many-body electron problem (where the electron-electron (e-e) interaction term couples the single-particle equations together) onto a single-body problem. This is done by considering the ground-state expectation value of some observable $\mathcal{O}$ as a function of the electron density $n(\mathbf{r})$. A successful minimization of the energy functional will yield the ground-state density $n^{(0)}(\mathbf{r})$ and thus all others ground-state observables. First, one considers an energy functional that does not explicitly have an e-e interaction but has instead some other effective single-particle potential V_{s} that "bathes" the system and is felt by every electron individually. The corresponding system of Schrodinger's equations for the electrons wavefunction φ_i (where i denotes the electron) is know as the *Kohn-Sham* (KS) equations. This effective potential is generally composed of three terms: some applied external potential V, an Hartree term V_H describing the e-e Coulomb repulsion and an *exchange-correlation potential* V_{xc} introduced as mean to include all the many-body interactions that are otherwise not accounted for. Of course, the exact form of V_{xc} is not known and it must be approximated. Since both V_H and V_{xc} depend on $n(\mathbf{r})$, which depends on φ_i, which in turn depend on V_s, the problem of solving the KS equation has to be done in a self-consistent, iterative, way. Usually one starts with an initial guess for $n(\mathbf{r})$, then calculates the corresponding V_s and then solves the KS equations for the φ_i's. From these, one can calculate a new density and start all over again. This procedure is then repeated until convergence is reached.

There are, for example, several other ground-state mean-field approaches who use instead the (time-orded) single-particle Green's functions (GF) G as their building block, as apposed to the single-particle density in DFT. In this perturbation theory formalism, the broad strokes are still very similar: one obtains the non-interacting electron propagator G_0 in a mean field approximation and introduces a self-energy term Σ as basically a black box that contains all the unaccounted many-body interactions. Then, through the so called *Hedin's iterative scheme* depicted in Fig.(1) and a series of approximations, one can calculate G and, from there, obtain all the ground-state properties. Following Hedin's equations in Fig.(1), one of the most common approximations, known as the GW, makes the initial ansatz $\Sigma \approx 0$ such that $G \approx G_0$, leading to $\Gamma \approx 1$ which circles back to the self energy becoming $\Sigma = GW$ (hence the name). The corresponding screened potential W contains some of the effects of the dielectric screening (or polarization in a quantum chemist language) established by the rearrangement of the remaining electrons as a response to the removal of one of the electrons. As a follow-up approximation, one usually considers only a first-order perturbation theory with the self-energy reading $\Sigma = G_0 W_0$.

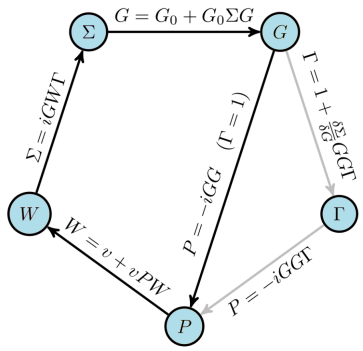

Figure 1. Hedin's iterative method composed of five, connected, integro-differential equation for the system's Green's function G, with G_0 its non-interacting counterpart, the irreducible vertex function Γ, the irreducible polarizability P, the screened Coulomb potential W and the self-energy Σ. The path made of black arrows shows the GW, process which bypasses the computation of Γ (gray arrows).

While DFT, GW and similar ground-state mean-field theories are extremely useful for ground-state properties they fail to attain the emergence of collective excitations from the residual interaction between the excited electron and the remaining hole. This should not surprise us since a variational mean-field theory is based on minimization [of the energy (at zero temperature) or of the free energy (at finite temperature)] and therefore it can only give reliable results for the quantities which we are minimizing. Since we are approximating a system of interacting particle to a system of independent particles, one would naively expect that a (mean-field) excited state could be obtained just by removing an electron from one of the occupied states $|o\rangle$ in the (mean-field) ground-state $|0\rangle_{\text{mf}}$, and promoting it to an empty state $|e\rangle$. Such excited state could then be written as $|e, o\rangle = c_e^\dagger c_o |0\rangle_{\text{mf}}$ and would have an energy equal to $\epsilon_e^{\text{mf}} - \epsilon_o^{\text{mf}} + E_0^{\text{mf}}$ with E_0^{mf} the (mean-field) ground-state energy. However this do not always apply since some states could have their energies bellow the quasi-particle (mean-field) bandgap. This is the case for excitons in insulators or plasmon-polaritons in metals. We emphasize that this inability in describing collective excitations is not a breakdown of mean-field theory, only of *ground-state* mean-field theory. If one instead describes the system using *time-dependent* mean-field theory this collective excitations do naturally emerge. Of course, there are other well established equivalent ways to describe this kind of collective excitation, for example via Configuration Interaction Singles (CIS), commonly employed in Quantum Chemistry, where we write the excited state as a linear combination $|X\rangle = \sum_{eo} \Psi_{eo}^X c_e^\dagger c_o |0\rangle_{\text{mf}}$ where Ψ_{eo}^X are the coefficients to be determined. As another example, one could obtain this kind of collective excitation via a time-dependent many-body perturbation theory, by resuming an infinite series of ladder and bubble (Feynman) diagrams in the two-particle propagator, which leads to the so called *Bethe-Salpeter equation* (BSE). Another possibility is using time-dependent Density Functional Theory (TD-DFT). For a comprehensive guide on all this formalism see Ref.[Bla20]. In particular, if one is interested in pedagogical discussion of the BSE for a two-particle Green's function see Ref.[Stri88].

B. Integration algorithms

1. *Schur's 1D integration method*

2. *Quantics tensor cross interpolation*

Quantics representation Consider some complicated function of many variables, having it's structure resolved in many different length scales, some long, some short. On one hand, long scales will

demand large domains of definitions, on another hand, short scales will demand dense grids. Putting these two together means that any numerical operation with our function (for example, integrating it) requires HUGE amounts of memory costs. So, can this exponential cost of memory be avoided without loss of accuracy? Yes, if functions *compressible!* Recently, two promising strategies have emerged to account for this accurate resolution with parsimonious memory usage, *quantics* and *tensor cross interpolation* (TCI). The quantics representation expresses functions as multi-index tensors with each index representing one bit of a binary encoding of one variable and then the TCI, if applicable, yields parsimonious interpolation of those multi-index tensors.

Let us consider a many-body wavefunction of a $\mathcal{D} = 2$ dimensional tight-binding spin system. Let $x \to i = 1, ..., \mathcal{N}$ and $y \to j = 1, 2, ..., \mathcal{N}$ be the discretized grid site's nomenclature and σ_{ij} the system's spin degree of freedom, such that at a given site indexed by ij, the spin state $|\sigma_{ij}\rangle$ can either be spin-up $|\uparrow\rangle \equiv |1\rangle$ or spin-down $|\uparrow\rangle \equiv |0\rangle$. Let us consider the notation where $\boldsymbol{\sigma}$ encompasses all the system's variables, and that, instead of the 2D grid ij indexing, we consider instead the mapping into a 1D grid i, i.e $\{ij\} = \{1, ..., \mathcal{N}\} \otimes \{1, ..., \mathcal{N}\} = \{11, ..., 1\mathcal{N}, 21, ..., 2\mathcal{N}, \mathcal{N}1, ..., \mathcal{N}\mathcal{N}\}$ to $\{\ell\} = \{1, ..., \mathcal{L}\}$ with $\mathcal{L} = \mathcal{D} \times \mathcal{N} = 2\mathcal{N}$. We describe such wavefunction as

$$|\Psi(\mathbf{r})\rangle = \sum_{\boldsymbol{\sigma}} f_{\boldsymbol{\sigma}} \, |\boldsymbol{\sigma}\rangle \tag{1}$$

where the probability of the system being in a certain configuration state $|\boldsymbol{\sigma}\rangle \equiv |\sigma_1 \sigma_2 ... \sigma_N\rangle \equiv |\sigma_1\rangle \otimes ... \otimes |\sigma_{\mathcal{L}}\rangle$ is dictated by $F_{\boldsymbol{\sigma}}$. So, for example, for a $\mathcal{N} \times \mathcal{N}$ grid with $\mathcal{N} = 2$ we have $\mathcal{L} = 4$ indexed sites/bits, meaning that the system can be in one of the $2^{\mathcal{N}} = 16$ configurations $\{|0000\rangle, |0001\rangle, |0010\rangle, ..., |1111\rangle\}$ with probabilities $\{f_{0000}, f_{0001}, f_{0010}, ..., f_{1111}\}$ respectively.

The main idea is that one can compress f as a product of matrix states (MPS), each encoding the information of a given site ℓ, yielding a factorized representation,

$$f_{\boldsymbol{\sigma}} \approx \mathbf{M}_1(\sigma_1)\mathbf{M}_2(\sigma_2)...\mathbf{M}_\ell(\sigma_\ell)...\mathbf{M}_{\mathcal{L}}(\sigma_{\mathcal{L}})$$

This compressed notation allows one to trivially calculate it's integral as a product of independent sums,

$$I^{(\mathcal{L})} = \int d\boldsymbol{\sigma} F_{\boldsymbol{\sigma}} \approx \sum_{\sigma_1} \mathbf{M}_1(\sigma_1) \sum_{\sigma_2} \mathbf{M}_2(\sigma_2) ... \sum_{\sigma_{\mathcal{L}}} \mathbf{M}_{\mathcal{L}}(\sigma_{\mathcal{L}}).$$

Tensor cross interpolation At this point one might be thinking "Alright, this seems pretty neat, but how does one actually do this compression?" The standard toolbox for compressing tensors is via a repeated *singular value decomposition* (SVD) where each matrix is factorized as $\mathbf{M}_\ell \approx \mathbf{U}_\ell \mathbf{S}_\ell \mathbf{V}_\ell^\dagger$ with $\mathbf{S}_\ell$ a diagonal sparse matrix of dimension $\mathcal{L} \times \mathcal{L}$. On the other hand, the dimension D_ℓ of the "virtual" bond indices α_ℓ does not need to always be $\mathcal{L}$ as this would be an incredible waste of memory, so one truncates D_ℓ at a given $D_{\max}$, by discarding singular values smaller than a specified truncation threshold ε. In this sense, the SVD decomposition method is known as a rank-revealing decomposition. This is a reasonable approximation if $D_{\max} \ll 2^{\mathcal{L}/2}$, implying that $F_{\boldsymbol{\sigma}}$ has a noticeable internal structure that can be encoded with a small information content. However, even if $F_{\boldsymbol{\sigma}}$ is strongly compressible, the SVD unfolding strategy, although optimally accurate, is exponentially inefficient. In another words, although it uncovers structure in $F_{\boldsymbol{\sigma}}$, it does not exploit it as it is constructing it as it requires knowledge of the full tensor $F_{\boldsymbol{\sigma}}$. So, to resume, SVD provides great accuracy but has exponentially growing memory cost $\mathcal{O}(2^{\mathcal{L}/2})$ and exponentially long runtimes $\mathcal{O}(2^{\mathcal{L}})$, which is not, by any mean, ideal.

Hence comes the tensor cross interpolation (TCI), a exponentially more efficient algorithm, memory and runtime wise. It needs at most $\mathcal{O}(D_{\max}^2 \mathcal{L})$ function evaluations and a runtime of at most $\mathcal{O}(D_{\max}^3 \mathcal{L})$, just with the small sacrifice of a tad bit of accuracy, requiring a slightly larger $D_{\max}$ for a specified error tolerance ε.

The TCI algorithm serves as a black box that samples $F_{\boldsymbol{\sigma}}$ at some clever choices of $\boldsymbol{\sigma}$ and iteratively constructs the TT from the sampled values. TCI achieves the factorization needed for unfolding by employing matrix cross interpolation (MCI) rather than SVD. The MCI formula approximates it as $\mathbf{M}_\ell \approx \mathbf{C}_\ell \mathbf{P}_\ell^{-1} \mathbf{R}_\ell$. The column, row and pivot matrices $\mathbf{C}$, $\mathbf{R}$, and $\mathbf{P}$, are all constructed from elements of $\mathbf{M}$: $\mathbf{C}$ contains D columns (red), $\mathbf{R}$ contains D rows (blue), and $\mathbf{P}$ their intersections, the pivots (purple). The resulting $\tilde{\mathbf{M}}$ exactly reproduces all elements of $\mathbf{M}$ contained in $\mathbf{C}$ and $\mathbf{R}$; the remaining (grey) elements are in effect interpolated from the "crosses" formed by these (hence "cross interpolation"). Understand that if the original matrix $\mathbf{M}$ is completely random then the interpolation will fail miserably, meaning that non-smooth functions will have it's issues. If $\mathbf{D} = \dim(\mathbf{M})$, one can obtain an exact representation of the full matrix, $\mathbf{M} = \tilde{\mathbf{M}}$.

It is important to note that MCI is not the numerical excellence peak of decomposition methods; as of today, that title belongs to *partial rank-revealing lower-upper* (prrLU) decomposition. Although mathematically equivalent to MCI, it is numerically more stable as the pivot matrices are never constructed nor inverted explicitly. The standard LU decomposition factorizes a matrix as $\mathbf{M} = \mathbf{LDU}$, where $\mathbf{L}$ is lower-triangular, $\mathbf{D}$ diagonal and $\mathbf{U}$ upper-triangular through the Gaussian elimination algorithm, known for inverting matrices or solving linear systems of equations. As a step up, the prrLU decomposition has two additional features, it is both rank-revealing—the largest remaining element, found by pivoting on both rows and columns, is used for the next pivot—, and partial—Gaussian elimination is stopped after constructing the first $\tilde{\chi}$ columns of $\mathbf{L}$ and rows of $\mathbf{U}$, such that $\mathbf{LDU}$ is a rank-$\tilde{\chi}$ factorization of $\mathbf{M}$.

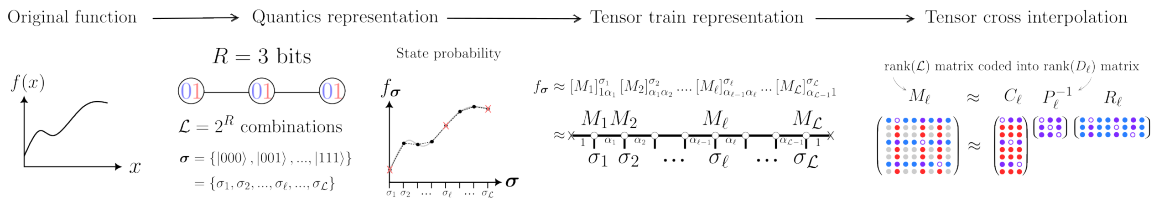

Figure 2. QTCI algorithm sketch. The red crosses on $f_{\boldsymbol{\sigma}}$ are to show that the TCI algorithm does not need to evaluate all points, only a selected few. For larger bit sizes $N_{TT} \lll \mathcal{L}$.

QTCI benchmarking for smooth, non-differentiable and discontinuous functions Here, we benchmark and test the QTCI codification algorithm (see a tutorial here [1]) for three different functions—smooth, non-differentiable and discontinuous functions—, in 1D and 2D. As we explained above, we know that the QTCI is very efficient to resolve structure in many different length scales. But how does it deal when this structure is not smooth? Will it find order in it's kinks and jumps? Or will it lose all efficiency? This become very relevant if one is, for example, dealing with systems at zero-temperature.

For the 2D tests we consider a normal distribution as our smooth function, a cone as our non-differentiable function and a cylinder as our discontinuous functions. For the 1D teste we consider the $y = 0.0$ linecuts of this functions.

Let us start with the 1D tests, which we expected better results since TT are a 1D dimensional object. In Fig.(3)(a) we show the

as shown in Fig.(3), allowing us to test how the algorithms handle kinks and discontinuities respectively. We benchmark the integrations first for a 1D case, picking some linecut of fixed y, and then the full 2D integration. As we can see from the benchmark of this simple examples in Fig.(3), for smaller tolerances, the quadGK algorithm with the problematic points given a priori, wins hcubature every time, by, at least, a factor of 10. Because of this we take the hcubature method out of the race and focus only on the quadGK (with the problematic points given a priori) and on the QTCI. One could think that from this results QTCI seem to be even worse than hcubature, but this is only because we are not using it right and mostly because we aren't even in the regime when it excels at, which is high

dimensional functions. So, for this, we examine in the next section how to really use the QTCI, and then compare it with quadGK.

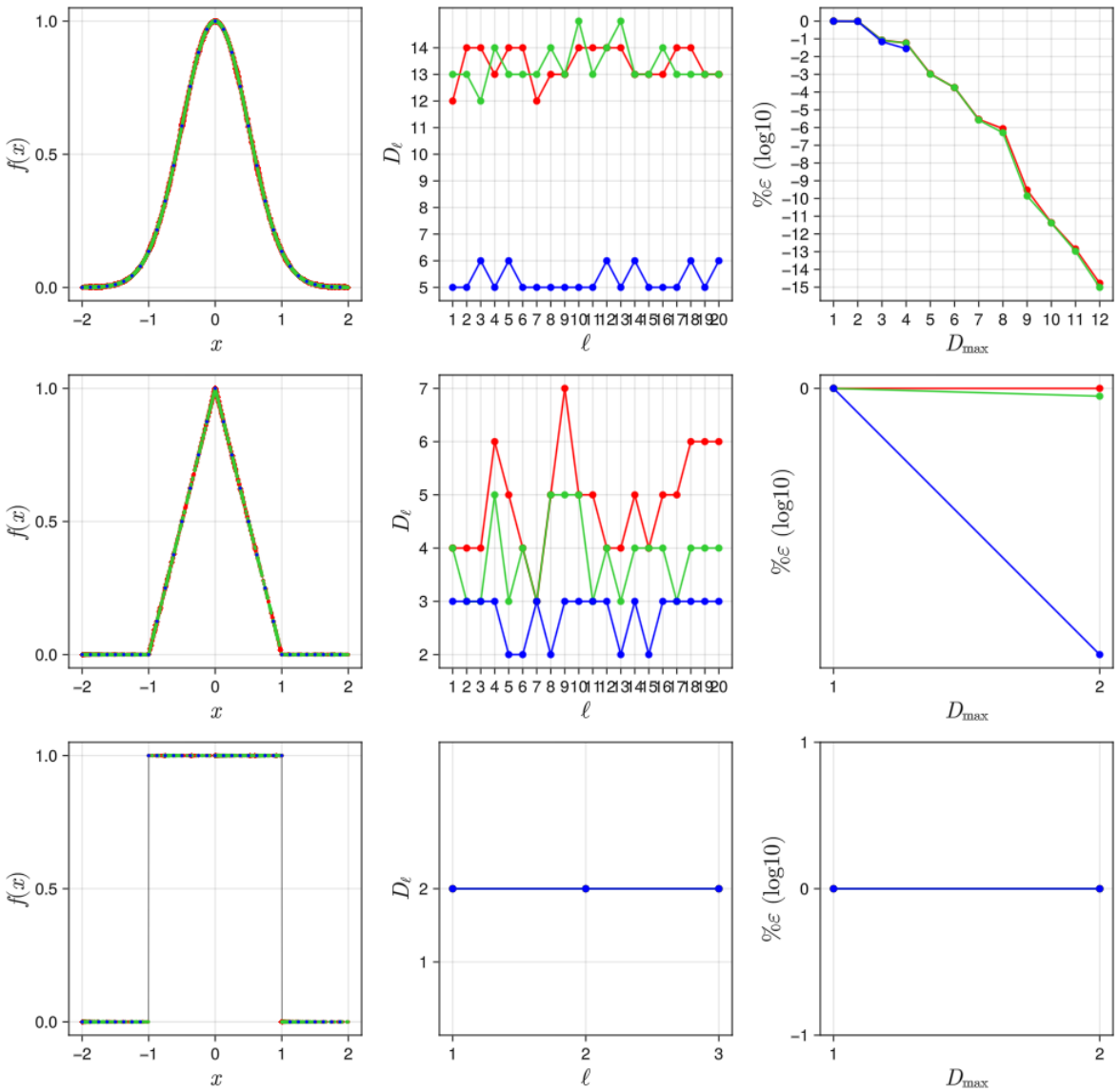

Figure 3. 1D QTCI benchmarking of a **(1)** Gaussian (smooth function), a **(2)** triangle (non-differentiable function), and a **(3)** square (discontinuous function). For the three cases, the panels **(a)** through **(c)** use a quantics grid of $R = 3$, 10 and 20 bits, at a total of $N_R = 2^R$ grid points, corresponding, respectively, to the QTCI reconstructed curves in red, green and blue. Moreover, the absolute tolerance for all curves is $\varepsilon_a = 10^{-10}$. In panels **(a)**, the scatter points in the different colors, at a total N_{QTT} for each color, are the points that the QTCI algorithm actually evaluated. For few bits $N_{QTT} \lessapprox N_R$ and for a lot of bits, ideally, $N_{QTT} \lll N_R$. The original function is shown in black. In panels **(b)** we plot the virtual bond dimensions D_ℓ of the QTT. In panels **(c)** we plot the relative error estimates as a function of $D_{\max} = \max(D_\ell)$, being capped at rank 20. In panels **(d)** we plot the time of QTCI as a function of the bits R and absolute tolerance ε_a tyo seek the

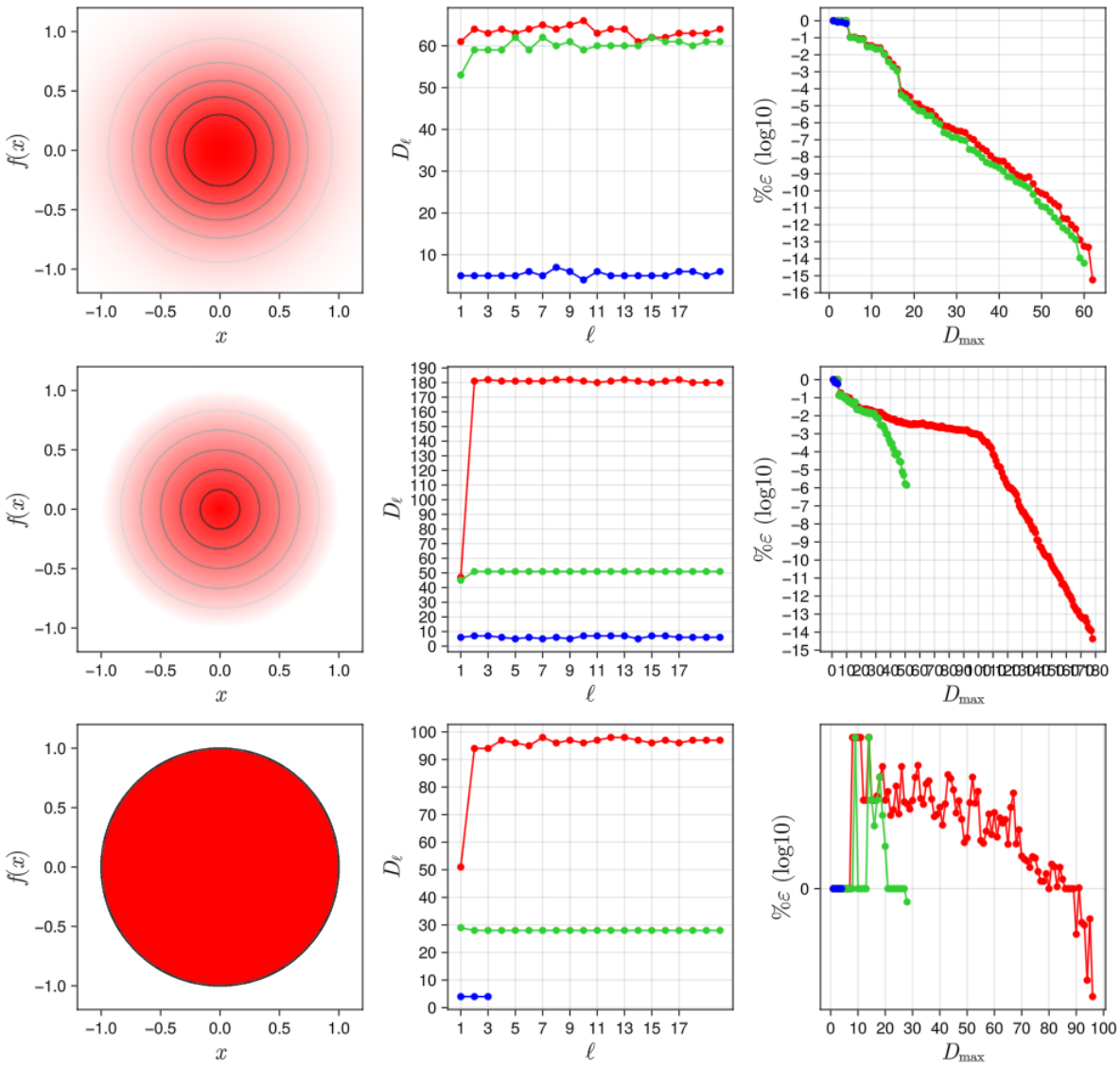

Figure 4. Same as Fig.(3) but in 2D and for $R = 3$, 6 and 9 bits with a a cap at rank 100.

3. *Benchmarking integration algorithms*

Here, we benchmark the efficiency of the different integration solvers in the eventuality of the functions having kinks and/or discontinuities. This will be very useful when deciding which of the solvers one should pick when integrating over the 1st Brillouin zone (1BZ) in order to obtain the density matrix of the system's unbounded Hamiltonian at a given site at zero temperature,

$$\rho_{ii} = \frac{1}{2\pi} \int_{1\mathrm{BZ}} d\mathbf{k} \sum_{n}^{\mathrm{bw}} f(\omega_{\mathbf{k}n}, \omega_f) \left|\mathbf{k}n\right\rangle \left\langle \mathbf{k}n\right| ,$$

where f is the Fermi-Dirac distribution, behaving as an Heaviside step-function at zero temperature at the Fermi energy ω_f, $\omega_{\mathbf{k}n}$ the eigen-energy at a given momentum point $\mathbf{k}$ and band n on the bandstructure and $|\mathbf{k}n\rangle$ the respective eigen-state.

In general, performing this type of integration numerically is a very intensive task and several methods have been developed to make it as efficient as possible. Here we highlight the main contenders: the Gauss–Kronrod's quadrature (QuadGK) [2], the Cubature integration [3] and the tensor cross interpolation (TCI) [4]. The main practical difference between QuadGK and Cubature is that Cubature can, by itself, find the problematic points (be it kinks or discontinuities) and focus its attention around them, always avoiding evaluating exactly the point. QuadGK, on the other hand, is very much dependent on these problematic points, but one can actively subdivide the integrals into segments where the exact endpoints (chosen as the problematic points) are ignored. As for the TCI method, the main idea is to codify (in a highly efficient way, mind you) the function we want to integrate into a so-called tensor train, which we then operate on. In general, this method is very promising because one could codify something for single set of parameters and then, with little to no work, build that something for all other set of parameters. What this means in practicality is that one could compute a phase diagrams in record time. Integration wise there are some downturns because TCI does not deal well with discontinuities (which are unavoidable at zero temperature). If you wish to know how what exactly are in the inner works of these algorithms, as well as the theoretical details, check [5] for QuadGK, [6] for Cubature and [7][8] as well as the next section for TCI.

Since this is just a benchmark of the different algorithms, we will consider simpler functions $\int_{-1}^{+1} d\mathbf{r} f(\mathbf{r})$, namely a normal distribution, a cone and a cylinder as shown in Fig.(5), allowing us to test how the algorithms handle kinks and discontinuities respectively. We benchmark the integrations first for a 1D case, picking some linecut of fixed y, and then the full 2D integration. As we can see from the benchmark of this simple examples in Fig.(5), for smaller tolerances, the quadGK algorithm with the problematic points given a priori, wins hcubature every time, by, at least, a factor of 10. Because of this we take the hcubature method out of the race and focus only on the quadGK (with the problematic points given a priori) and on the QTCI. One could think that from this results QTCI seem to be even worse than hcubature, but this is only because we are not using it right and mostly because we aren't even in the regime when it excels at, which is high dimensional functions. So, for this, we examine in the next section how to really use the QTCI, and then compare it with quadGK.

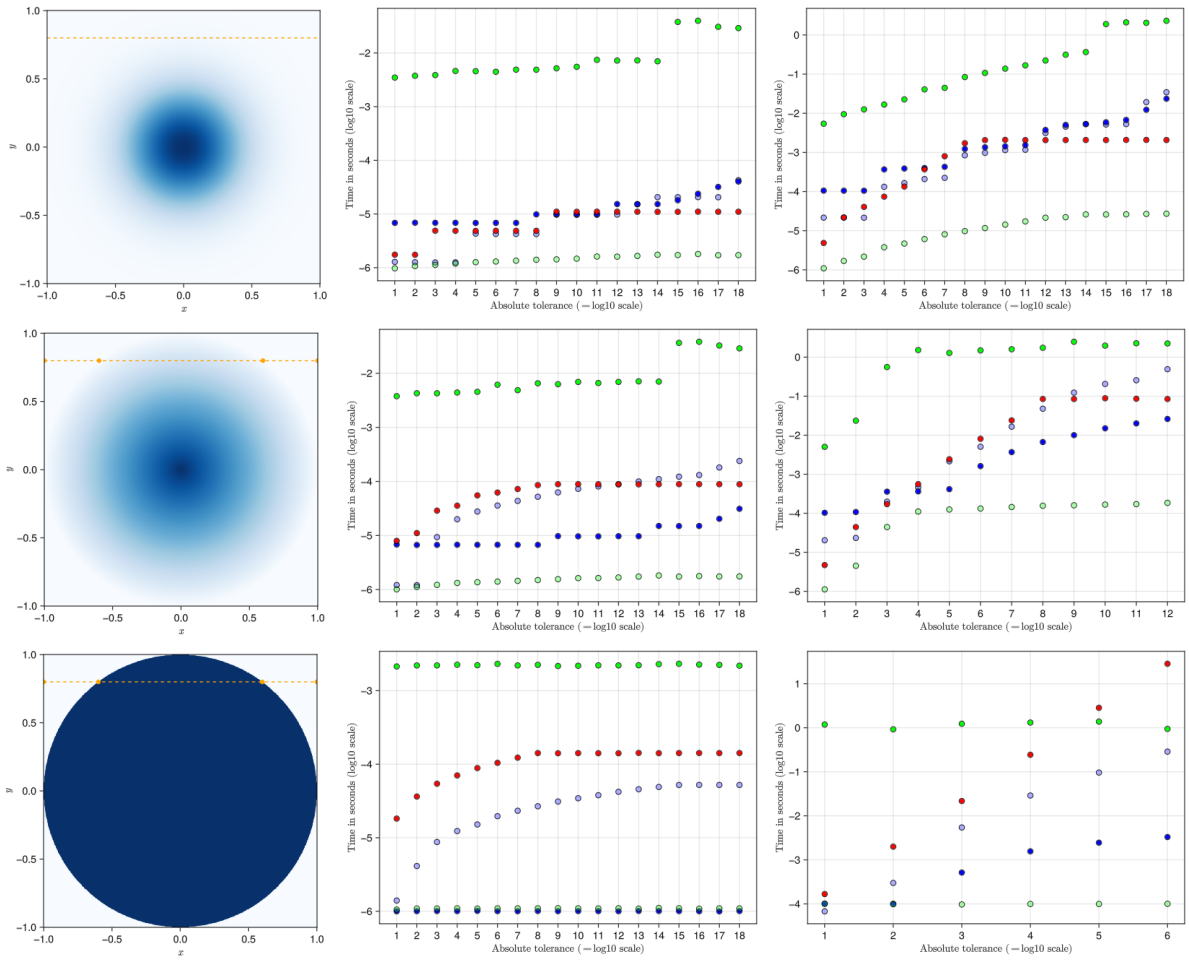

Figure 5. From left to right: **(top)** gaussian, **(center)** cone and **(bottom)** cylinder integration benchmark times in 1D (along the orange dashed linecut) and in 2D. As for the methods of integrations we have in (red) hcubature, (green) quantics TCI interpolation *plus* integration times, with transparent green jst the integration time, and (blue) quadGK, with transparent blue without feeding it the discontinuity/kink points

[1] Https://tensor4all.org/T4AJuliaTutorials/ipynbs/quantics1d.html#.
[2] Https://juliamath.github.io/QuadGK.jl/stable/gauss-kronrod/.
[3] Https://github.com/JuliaMath/Cubature.jl.
[4] Https://tensor4all.org/TensorCrossInterpolation.jl/dev/.
[5] ??
[6] ??
[7] Phys. Rev. Lett. 132, 056501.
[8] ArXiv:2407.02454 [physics.comp-ph].